

1. DFS – 8 PUZZLE PROBLEM

```
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)
```

```
def print_state(state):
```

```
    for i in range(0, 9, 3):
```

```
        row = state[i:i+3]
```

```
        print(' '.join(str(x) if x != 0 else '_' for x in row))
```

```
    print()
```

```
def get_successors(state):
```

```
    successors = []
```

```
    blank = state.index(0)
```

```
    row, col = divmod(blank, 3)
```

```
    moves = {
```

```
        'Up': (row - 1, col),
```

```
        'Down': (row + 1, col),
```

```
        'Left': (row, col - 1),
```

```
        'Right': (row, col + 1)
```

```
    }
```

```
    for move_name, (new_row, new_col) in moves.items():
```

```
        if 0 <= new_row < 3 and 0 <= new_col < 3:
```

```
            new_blank = new_row * 3 + new_col
```

```
            new_state = list(state)
```

```
            new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
```

```
            successors.append((move_name, tuple(new_state)))
```

```
    return successors
```

```

def is_solvable(state):
    # Count inversions (ignoring the blank tile); even count => solvable
    tiles = [t for t in state if t != 0]
    inversions = 0
    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] > tiles[j]:
                inversions += 1
    return inversions % 2 == 0

def dfs_8puzzle(start_state, max_depth=30):
    stack = [(start_state, [start_state], [])]
    visited = {start_state}

    while stack:
        state, path, moves = stack.pop()

        if state == GOAL_STATE:
            return path, moves

        if len(moves) >= max_depth:
            continue

        for move_name, successor in get_successors(state):
            if successor not in visited:
                visited.add(successor)
                stack.append((successor, path + [successor], moves + [move_name]))

    return None, None

```

```

def read_state(prompt):
    print(prompt + " (enter 9 numbers 0-8, 0 = blank, space-separated, row by row)")
    while True:
        try:
            values = list(map(int, input("> ").split()))
            if sorted(values) == list(range(9)):
                return tuple(values)
            print("Invalid! Enter each of 0-8 exactly once.")
        except ValueError:
            print("Please enter 9 valid integers.")

if __name__ == "__main__":
    print("=== 8-Puzzle Solver using DFS ===\n")
    start_state = read_state("Enter the initial puzzle state")

    print("\nInitial State:")
    print_state(start_state)
    print("Goal State:")
    print_state(GOAL_STATE)

    if not is_solvable(start_state):
        print("This puzzle configuration is NOT solvable.")
    else:
        path, moves = dfs_8puzzle(start_state)

        if path is None:
            print("No solution found within the depth limit explored by DFS.")
        else:
            print(f"Solution found by DFS in {len(moves)} moves!\n")
            print("Move sequence:", moves)

```

```

print("\nStep-by-step states:\n")

for step, state in enumerate(path):

    print(f"Step {step}:")

    print_state(state)

```

OUTPUT :

```

Enter the initial puzzle state (enter 9 numbers 0-8, 0 = blank, space-separated
, row by row)
>
  1 2 3 4 0 6 7 5 8

Initial State:
1 2 3
4 _ 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 _

Solution found by DFS in 22 moves!

Move sequence: ['Right', 'Down', 'Left', 'Left', 'Up', 'Right', 'Right', 'Up',
'Left', 'Down', 'Left', 'Down', 'Right', 'Up', 'Up', 'Right', 'Down', 'Down', '
Left', 'Up', 'Right', 'Down']

Step-by-step states:

Step 0:
1 2 3
4 _ 6
7 5 8

Step 1:
1 2 3
4 6 _
7 5 8

Step 2:
1 2 3
4 6 8
7 5 _

```

Step 16:

1 2 _
4 8 3
7 6 5

Step 17:

1 2 3
4 8 _
7 6 5

Step 18:

1 2 3
4 8 5
7 6 _

Step 19:

1 2 3
4 8 5
7 _ 6

Step 20:

1 2 3
4 _ 5
7 8 6

Step 21:

1 2 3
4 5 _
7 8 6

Step 22:

1 2 3
4 5 6
7 8 _

2. IDS – 8 PUZZLE PROBLEM

```
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)
```

```
def print_state(state):
```

```
    for i in range(0, 9, 3):
```

```
        row = state[i:i+3]
```

```
        print(' '.join(str(x) if x != 0 else '_' for x in row))
```

```
    print()
```

```
def get_successors(state):
```

```
    successors = []
```

```
    blank = state.index(0)
```

```
    row, col = divmod(blank, 3)
```

```
    moves = {
```

```
        'Up': (row - 1, col),
```

```
        'Down': (row + 1, col),
```

```
        'Left': (row, col - 1),
```

```
        'Right': (row, col + 1)
```

```
    }
```

```
    for move_name, (new_row, new_col) in moves.items():
```

```
        if 0 <= new_row < 3 and 0 <= new_col < 3:
```

```
            new_blank = new_row * 3 + new_col
```

```
            new_state = list(state)
```

```
            new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
```

```
            successors.append((move_name, tuple(new_state)))
```

```
    return successors
```

```

def is_solvable(state):
    tiles = [t for t in state if t != 0]
    inversions = 0
    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] > tiles[j]:
                inversions += 1
    return inversions % 2 == 0


def depth_limited_search(state, depth_limit, path, moves, visited):
    # Goal test
    if state == GOAL_STATE:
        return path, moves

    # Depth limit reached -> dead end for this iteration
    if depth_limit == 0:
        return None, None

    visited.add(state)

    for move_name, successor in get_successors(state):
        if successor not in visited:
            result_path, result_moves = depth_limited_search(
                successor, depth_limit - 1,
                path + [successor], moves + [move_name],
                visited
            )
            if result_path is not None:
                return result_path, result_moves

```

```
visited.discard(state) # allow revisiting on a different branch/iteration

return None, None
```

```
def ids_8puzzle(start_state, max_depth_limit=30):
    for depth_limit in range(max_depth_limit + 1):
        print(f"Trying depth limit = {depth_limit} ...")
        visited = set()

        path, moves = depth_limited_search(start_state, depth_limit, [start_state], [], visited)

        if path is not None:
            return path, moves, depth_limit
    return None, None, None
```

```
def read_state(prompt):
    print(prompt + " (enter 9 numbers 0-8, 0 = blank, space-separated, row by row)")
    while True:
        try:
            values = list(map(int, input("> ").split()))
            if sorted(values) == list(range(9)):
                return tuple(values)
            print("Invalid! Enter each of 0-8 exactly once.")
        except ValueError:
            print("Please enter 9 valid integers.")
```

```
if __name__ == "__main__":
    print("=== 8-Puzzle Solver using Iterative Deepening Search (IDS) ===\n")
    start_state = read_state("Enter the initial puzzle state")
```

```
print("\nInitial State:")
print_state(start_state)
print("Goal State:")
print_state(GOAL_STATE)
```

```
if not is_solvable(start_state):
    print("This puzzle configuration is NOT solvable.")
else:
    path, moves, solved_depth = ids_8puzzle(start_state)
```

```
if path is None:
    print("No solution found within the max depth limit.")
```

```
else:
    print(f"\nSolution found by IDS at depth {solved_depth} ({len(moves)} moves)!\n")
    print("Move sequence:", moves)
    print("\nStep-by-step states:\n")
    for step, state in enumerate(path):
        print(f"Step {step}:")
        print_state(state)
```

OUTPUT :

```
Enter the initial puzzle state (enter 9 numbers 0-8, 0 = blank, space-separated, row by row)
> 1 2 3 4 0 6 7 5 8
```

Initial State:

```
1 2 3
4 _ 6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 _
```

Trying depth limit = 0 ...

Trying depth limit = 1 ...

Trying depth limit = 2 ...

Solution found by IDS at depth 2 (2 moves)!

Move sequence: ['Down', 'Right']

Step-by-step states:

Step 0:

```
1 2 3
4 _ 6
7 5 8
```

Step 1:

```
1 2 3
4 5 6
7 _ 8
```

Step 2:

```
1 2 3
4 5 6
7 8 _
```